

BCC202 - Estruturas de Dados I

Aula 25: Tabelas *Hash* - Endereçamento Aberto

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Abordagens de pesquisa em Memória Primária

- ▶ Pesquisa Sequencial.
- ▶ Pesquisa Binária.
- ▶ Árvores de Pesquisa:
 - ▶ Árvores Binárias de Pesquisa.
 - ▶ Árvores AVL.
- ▶ Transformação de Chave (*Hashing*):
 - ▶ *Hashing* Perfeito.
 - ▶ Listas Encadeadas.
 - ▶ Endereçamento Aberto.

Conteúdo

Introdução

Funções de Transformação

Endereçamento Aberto

Considerações Finais

Bibliografia

Introdução

Hash significa:

- ▶ Fazer picadinho de carne e vegetais para cozinhar.
- ▶ Fazer uma bagunça. (**Webster's New World Dictionary**)
- ▶ Espalhar x Transformar (informática x computação).

Em uma Tabela *Hash*, os registros armazenados em um vetor são diretamente endereçados a partir de uma **transformação aritmética e/ou lógica sobre a chave de pesquisa**.

Tabela Hash

- ▶ Estrutura especial de dados especial.
- ▶ Armazena as informações desejadas associando a uma **chave**.
- ▶ Cada posição pode armazenar uma lista ou somente uma **informação**.
- ▶ *Hashing* Perfeito $\times$ *Hashing* Imperfeito.

Transformação de Chave (*Hashing*)

Um método de pesquisa com o uso da transformação de chave é constituído de duas etapas principais:

1. Computar o valor da **função de transformação**, a qual transforma a chave de pesquisa em um endereço da tabela.
2. Considerando que duas ou mais chaves podem ser transformadas em um mesmo endereço de tabela, é necessário existir um método para lidar com **colisões**.

Funções de Transformação

Funções de Transformação

- ▶ Uma função de transformação deve **mapear** chaves em inteiros dentro do intervalo $[0, \dots, M - 1]$, onde M é o tamanho da tabela.
- ▶ A função de transformação ideal é aquela que:
 - ▶ Seja **simples** de ser computada.
 - ▶ Para cada chave de entrada, qualquer uma das saídas possíveis é **igualmente provável de ocorrer**.

Função de Transformação mais usada

- Usa o resto da divisão por M :

$$h(K) = K \% M \quad (\text{em linguagem C})$$

onde K é um inteiro correspondente à chave e M é o tamanho da tabela.

Transformações de Chaves não numéricas

- ▶ As chaves **não numéricas** devem ser transformadas em números:

$$K = \sum_{i=1}^n Chave[i] \times p[i],$$

onde:

- ▶ n é o número de caracteres da chave.
- ▶ $Chave[i]$ corresponde à representação ASCII do i -ésimo caractere da chave.
- ▶ $p[i]$ é um inteiro de um conjunto de pesos gerados randomicamente para $1 \leq i \leq n$.

Implementação da função de transformação para Chaves não numéricas

```
1 /* Funcao de hash que retorna o indice (numero inteiro)
2  * de uma chave (string) */
3 int h(char *chave, int p[], int m, int tam_p) {
4     unsigned int soma = 0;
5     int i, stringSize = strlen(chave);
6
7     for (i = 0; i < stringSize; i++)
8         soma += (unsigned int) chave[i] * p[i % tam_p];
9
10    return (soma % m);
11 }
```

Resolvendo Colisões

Formas de tratar a colisão:

- ▶ Listas Encadeadas.
- ▶ **Endereçamento Aberto.**

Endereçamento Aberto

Endereçamento Aberto

- ▶ Quando o número de registros a serem armazenados na tabela puder ser **previamente estimado**, então não haverá necessidade de usar apontadores para armazenar os registros.
- ▶ Existem vários métodos para armazenar N registros em uma tabela de tamanho $M > N$, os quais utilizam os lugares vazios na própria tabela para resolver as colisões. (Knuth, 1973, p.518)

Endereçamento Aberto

- ▶ No **Endereçamento Aberto** todas as chaves são armazenadas na própria tabela, sem o uso de ponteiros explícitos.
- ▶ Existem várias propostas para a escolha de localizações alternativas. A mais simples é chamada de *hashing* linear, onde a posição h_j na tabela é dada por:

$$h_j = (h(x) + j) \text{ mod } M, \text{ para } 0 \leq j \leq M - 1.$$

Inserção em uma *Hash* de Endereçamento Aberto

- ▶ Se a i -ésima letra do alfabeto é representada pelo número i e a função de transformação

$$h(chave) = chave \bmod M$$

é utilizada para $M = 7$, teremos:

- ▶ $h(L) = h(12) = 5$
- ▶ $h(U) = h(21) = 0$
- ▶ $h(N) = h(14) = 0$
- ▶ $h(E) = h(5) = 5$
- ▶ $h(S) = h(19) = 5$

Exemplo

Inserção de:

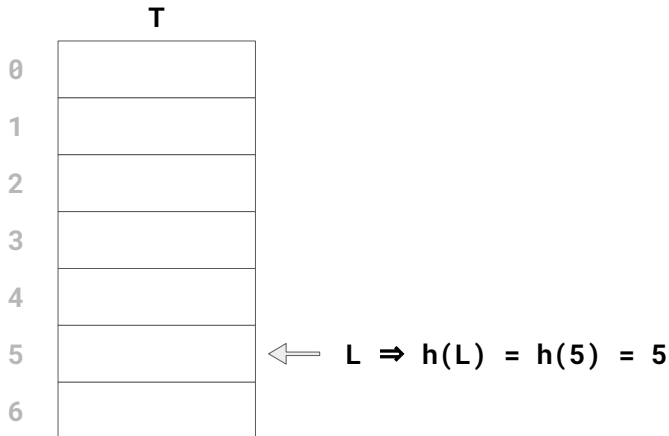
► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$



Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	
1	
2	
3	
4	
5	L
6	

Exemplo

Inserção de:

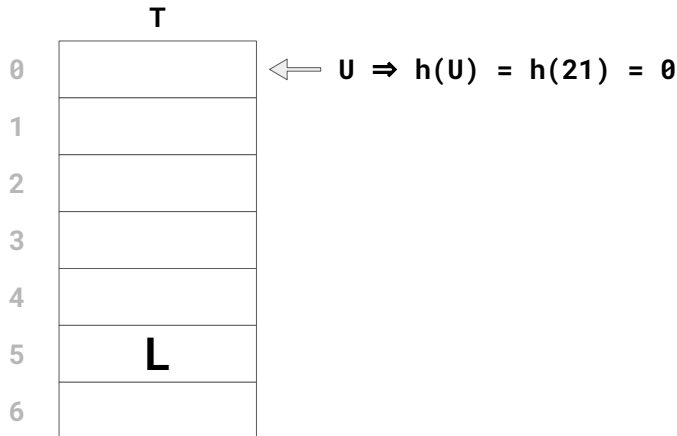
► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$



Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	
2	
3	
4	
5	L
6	

Exemplo

Inserção de:

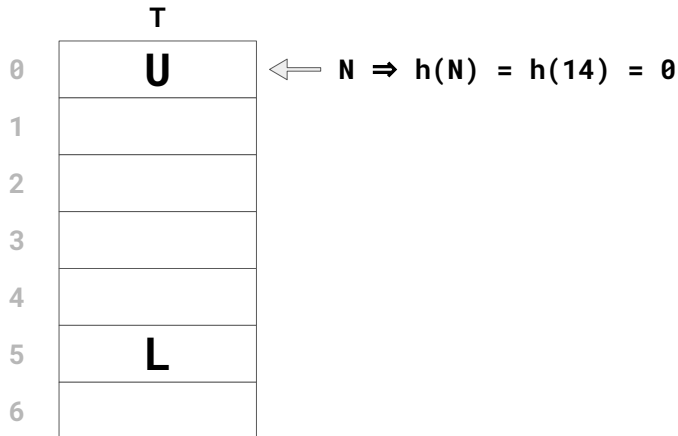
► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$



Exemplo

Inserção de:

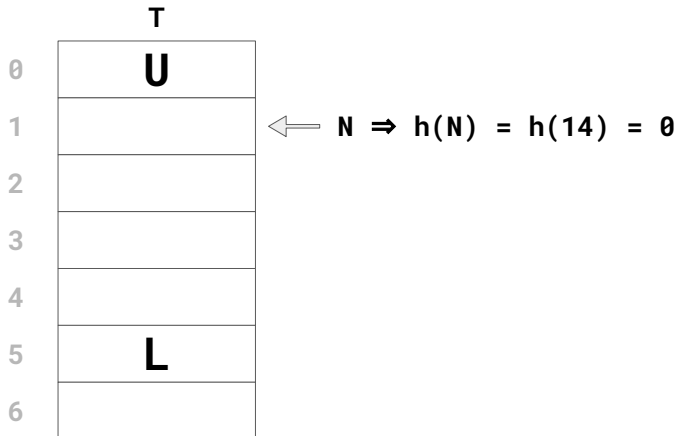
► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$



Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	

← $E \Rightarrow h(E) = h(5) = 5$

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	

← $E \Rightarrow h(E) = h(5) = 5$

Exemplo

Inserção de:

- ▶ $h(L) = h(12) = 5$
- ▶ $h(U) = h(21) = 0$
- ▶ $h(N) = h(14) = 0$
- ▶ $h(E) = h(5) = 5$
- ▶ $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	E

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	E

← $S \Rightarrow h(S) = h(19) = 5$

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	E

← $S \Rightarrow h(S) = h(19) = 5$

Exemplo

Inserção de:

- ▶ $h(L) = h(12) = 5$
- ▶ $h(U) = h(21) = 0$
- ▶ $h(N) = h(14) = 0$
- ▶ $h(E) = h(5) = 5$
- ▶ $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	E

← $S \Rightarrow h(S) = h(19) = 5$

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	E

← S ⇒ $h(S) = h(19) = 5$

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	
3	
4	
5	L
6	E

← $S \Rightarrow h(S) = h(19) = 5$

Exemplo

Inserção de:

► $h(L) = h(12) = 5$

► $h(U) = h(21) = 0$

► $h(N) = h(14) = 0$

► $h(E) = h(5) = 5$

► $h(S) = h(19) = 5$

T	
0	U
1	N
2	S
3	
4	
5	L
6	E

Dicionário usando Endereçamento Aberto

```
1 #define VAZIO "!!!!!!!!!!\0"
2 #define N 10 // tamanho da chave (string)
3 #define M 100 // tamanho da tabela
4
5 typedef char Chave[N];
6
7 typedef struct {
8     /* outros componentes */
9     Chave Chave;
10 } Item;
11
12 typedef Item Dicionario[M];
```

Inicializando um Dicionário de Endereçamento Aberto

```
1 void DicionarioInicia(Dicionario dic) {  
2     int i;  
3     for (i = 0; i < M; i++) {  
4         memcpy(dic[i].chave, VAZIO, N);  
5     }  
6 }
```

Pesquisando em um Dicionário de Endereçamento Aberto

```
1 int DicionarioPesquisa(Dicionario dic, Chave chave, int p) {
2     int j = 0;
3     int ini = h(chave, p);
4
5     while (strcmp(dic[(ini + j) % M].chave, VAZIO) != 0 &&
6           strcmp(dic[(ini + j) % M].chave, chave) != 0 &&
7           j < M) {
8         j++;
9     }
10
11     if (strcmp(dic[(ini + j) % M].chave, chave) == 0) {
12         return (ini + j) % M;
13     }
14
15     return -1; // pesquisa sem sucesso
16 }
```

Inserção de um item no Dicionário de Endereçamento Aberto

```
1 int DicionarioInsere(Dicionario dic, Item x, int p) {
2     if (DicionarioPesquisa(dic, x.chave, p) >= 0)
3         return 0; // chave já existe no dicionário
4
5     int j = 0;
6     int ini = h(x.chave, p);
7     while (strcmp(dic[(ini + j) % M].chave, VAZIO) != 0 && j < M) {
8         j++;
9     }
10
11     if (j < M) {
12         dic[(ini + j) % M] = x;
13         return 1;
14     }
15     return 0;
16 }
```

Remoção de um item do Dicionário de Endereçamento Aberto

```
1 int DicionarioRetira(Dicionario dic, Item *x, int p) {  
2     int i = DicionarioPesquisa(dic, x->chave, p);  
3     if (i == -1) {  
4         return 0; // chave não encontrada  
5     }  
6  
7     memcpy(dic[i].chave, VAZIO, N);  
8     return 1;  
9 }
```

Análise de *Hashs* implementadas com Endereçamento Aberto

- ▶ Seja $\alpha = N/M$ o fator de carga da tabela. Conforme demonstrado por Knuth (1973), o custo de uma pesquisa com sucesso é:

$$C(n) = \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

- ▶ O *hashing* linear sofre de um mal chamado agrupamento (*clustering*) (Knuth, 1973, pp.520–521).
 - ▶ Este fenômeno ocorre na medida em que a tabela começa a ficar cheia, pois a inserção de uma nova chave tende a ocupar uma posição na tabela que esteja contígua a outras posições já ocupadas.

Análise de *Hashs* implementadas com Endereçamento Aberto

- ▶ Apesar do *hashing* linear ser um método relativamente “**pobre**” para resolver colisões os resultados apresentados são bons.
- ▶ O **melhor caso**, assim como o caso médio, é $O(1)$.
- ▶ Qual o pior caso?

Análise de *Hashs* implementadas com Endereçamento Aberto

- ▶ Apesar do *hashing* linear ser um método relativamente “**pobre**” para resolver colisões os resultados apresentados são bons.
- ▶ O **melhor caso**, assim como o caso médio, é $O(1)$.
- ▶ Qual o pior caso?
 - ▶ Quando é necessário percorrer toda a tabela para encontrar uma chave (excesso de colisões).
 - ▶ Complexidade do pior caso: $O(n)$.

Vantagens e Desvantagens

Vantagens:

- ▶ Alta eficiência no custo de pesquisa, que é $O(1)$ para o caso médio.
- ▶ Simplicidade de implementação.

Desvantagens:

- ▶ Custo para recuperar os registros ordenados pela chave é alto, sendo necessário ordenar toda a tabela.
- ▶ Pior caso é $O(n)$.

Considerações Finais

Conclusão

- ▶ Duas implementações para *Hash*: Lista encadeada e Endereçamento Aberto.
- ▶ São armazenados em Tabelas *Hash* uma coleção de registros com chaves.
- ▶ A posição de inserção de um novo registro depende do valor resultante da função *Hash*.
- ▶ A escolha do que fazer quando há uma colisão, depende da estratégia de implementação.
- ▶ Geralmente, procurar por uma chave é rápido.

Exercício

Mostre o passo-a-passo de como seria a inserção do seu nome onde cada letra é uma chave na Tabela *Hash* que utiliza **Endereçamento Aberto**.

A = h(1)	B = h(2)	C = h(3)	D = h(4)
E = h(5)	F = h(6)	G = h(7)	H = h(8)
I = h(9)	J = h(10)	K = h(11)	L = h(12)
M = h(13)	N = h(14)	O = h(15)	P = h(16)
Q = h(17)	R = h(18)	S = h(19)	T = h(20)
U = h(21)	V = h(22)	W = h(23)	X = h(24)
Y = h(25)	Z = h(26)		

Considere a função *Hash* e uma Tabela *Hash* de tamanho 70 ($M = 70$):

$$h(chave) = chave \underline{\text{mod}} M$$

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:

-  Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.
Introduction to Algorithms.
The MIT Press, 2011.